

WEP40 Wireless Packet Decryption and Aircrack Forensics

OJE DOMINION MAYOWA

FWSD2511507

Computer and Digital Forensics

MR. Aminu Idris

25th September 2026

LAB OVERVIEW

This lab examined WEP wireless encryption to understand its practical weaknesses. I analyzed 802.11 frame types, confirmed WEP-protected traffic and identified repeated IVs, then used aircrack-ng to recover the WEP key and airdecap-ng to decrypt the capture. From the decrypted traffic, I extracted endpoints, conversations, and IP-to-MAC mappings to reconstruct the network activity.

OBJECTIVES

- Identify management, control and data frames in an 802.11 capture.
- Explain WEP40 structure, RC4 keystream use, IV reuse and integrity limitations.
- Use Aircrack-ng on a supplied historical capture only.
- Decrypt WEP traffic using airdecap-ng and verify output files.
- Extract IP/MAC endpoints, protocols, images and HTML from decrypted traffic.
- Distinguish a WEP key from a human passphrase and document optional CTF password analysis.
- Recommend modern wireless security controls.

TOOLS

- Authorized training Evidence
- Kali Linux
- Microsoft Word
- Aircrack-ng Suite
- Md5sum and sha256sum
- foremost, binwalk, file, strings
- Wireshark and tshark
- xz-utils

METHODOLOGY

Lab Folder Structure and Evidence Preparation

```
Session Actions Edit View Help
(kali@kali)-[~]
$ cd SBT-DF203-Lab9
(kali@kali)-[~/SBT-DF203-Lab9]
$ ls
evidence  exported  reports  screenshots  scripts  working
(kali@kali)-[~/SBT-DF203-Lab9]
$ mkdir -p ~/SBT-DF203-Lab9/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab9
pwd
/home/kali/SBT-DF203-Lab9
find . -maxdepth 1 -type d -print
./working
./screenshots
./evidence
./scripts
./reports
./exported
(kali@kali)-[~/SBT-DF203-Lab9]
$
```

Fig 1: Setting up the lab Folder

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ sudo apt install -y aircrack-ng wireshark tshark foremost binwalk xz-utils

[sudo] password for kali:
wireshark is already the newest version (4.6.6-1).
tshark is already the newest version (4.6.6-1).
xz-utils is already the newest version (5.8.3-1).
The following packages were automatically installed and are no longer required:
  libavc1394-0  libmbcrypto16  libSDL2-classic  libvidstab1.1  openjdk-21-jre-headless  python3.13  python3.13-venv
  libicu76      libpostal-data  libsimdutf31    libxmlsec1-1   postgresql-common-dev  python3.13-dev  rpcsvc-proto
  libio-pty-perl  libpostal1      libteamctl0     libxmlsec1-openssl1  python3-pluginbase     python3.13-minimal
  libipc-run-perl  libpython3.13-dev  libunibreak6   openjdk-21-jre  python3-pysnmp4        python3.13-tk
Use 'sudo apt autoremove' to remove them.

Upgrading:
  aircrack-ng  binwalk  python3-binwalk

Installing:
  foremost

Summary:
  Upgrading: 3, Installing: 1, Removing: 0, Not Upgrading: 1411
  Download size: 727 kB
  Space needed: 108 kB / 59.8 GB available

Get:1 http://http.kali.org/kali kali-rolling/main amd64 aircrack-ng amd64 1:1.7+git20230807.4bf83f1a-3 [557 kB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 python3-binwalk all 2.4.3+dfsg1-3 [122 kB]
Get:3 http://http.kali.org/kali kali-rolling/main amd64 binwalk all 2.4.3+dfsg1-3 [6,008 B]
```

Fig 2: Setting up Lab Environment

Then we download the evidence source from the github link provided by the material.

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ wget -O evidence/file.xz \
'https://raw.githubusercontent.com/ctfs/write-ups-2015/master/codegate-ctf-2015/programming/good-crypto/file.xz'
--2026-09-28 14:12:33-- https://raw.githubusercontent.com/ctfs/write-ups-2015/master/codegate-ctf-2015/programming/good-crypto/file.xz
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 12837660 (12M) [application/octet-stream]
Saving to: 'evidence/file.xz'

evidence/file.xz      100%[=====>] 12.24M  646KB/s  in 16s

2026-09-28 14:12:54 (797 KB/s) - 'evidence/file.xz' saved [12837660/12837660]

(kali@kali)-[~/SBT-DF203-Lab9]
$
```

Fig 3: Downloaded Source Evidence

Then we check the hash value of the Source evidence to be able to get the Integrity of the Evidence.

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ sha256sum evidence/file.xz | tee reports/file_xz_sha256.txt
dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6  evidence/file.xz

(kali@kali)-[~/SBT-DF203-Lab9]
$
```

Fig 4: Sha256sum value of the Source Evidence

Then next, we make a working copy to preserve the Integrity of the Evidence

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ cp --preserve=timestamps evidence/file.xz working/file_working.xz

(kali@kali)-[~/SBT-DF203-Lab9]
$
```

Fig 5: Created a working copy for the Source Evidence

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ unxz -k working/file_working.xz
unxz: working/file_working: File exists

(kali@kali)-[~/SBT-DF203-Lab9]
$
```

Fig 6: Decompress the Working file

Then we capture the working file by inspecting its content using the file command

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ file working/file_working
working/file_working: pcap capture file, microsecond ts (little-endian) - version 2.4 (802.11, capture length 65535)

(kali@kali)-[~/SBT-DF203-Lab9]
$
```

Fig 7: Created a working file

Getting the hash values of both the raw file and the decompressed one

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ sha256sum working/file_working.xz working/file_working | tee reports/working_hashes.txt
dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6  working/file_working.xz
c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d  working/file_working

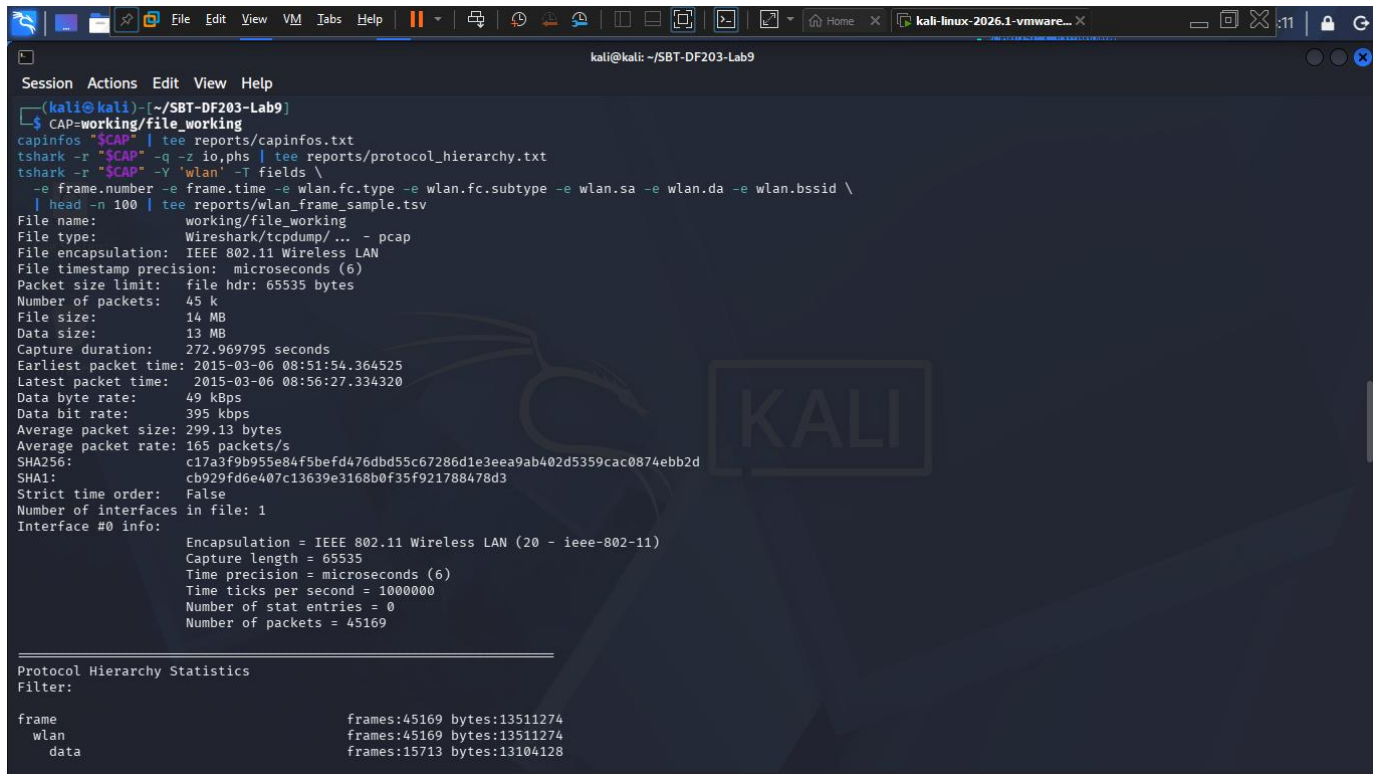
(kali@kali)-[~/SBT-DF203-Lab9]
$
```

Fig 8: The Hash Value of the raw file and the decompressed one

Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab9-DOMINION-OJE
Trainee name	OJE DOMINION MAYOWA
Date and time started	25/09/2026, 5:00PM
Evidence file name(s)	evidence/file.xz
Source or generation method	wget -O evidence/file.xz \ 'https://raw.githubusercontent.com/ctfs/write-ups-2015/master/codegate-ctf-2015/programming/good-crypto/file.xz'
Original SHA-256	dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6
Working-copy SHA-256	dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6
Analysis workstation/VM	KALI VM
Notes on any changes	

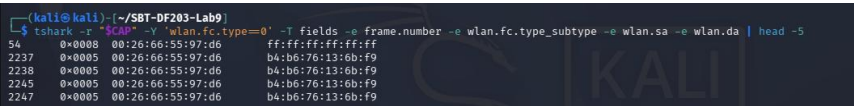
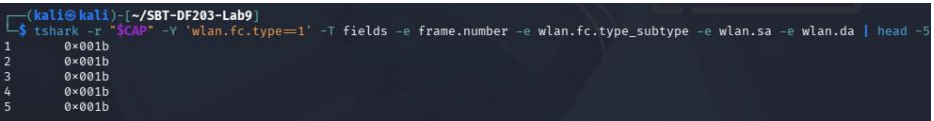
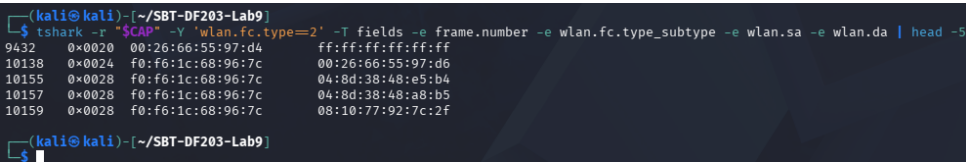
Part A - Inventory the Wireless Capture



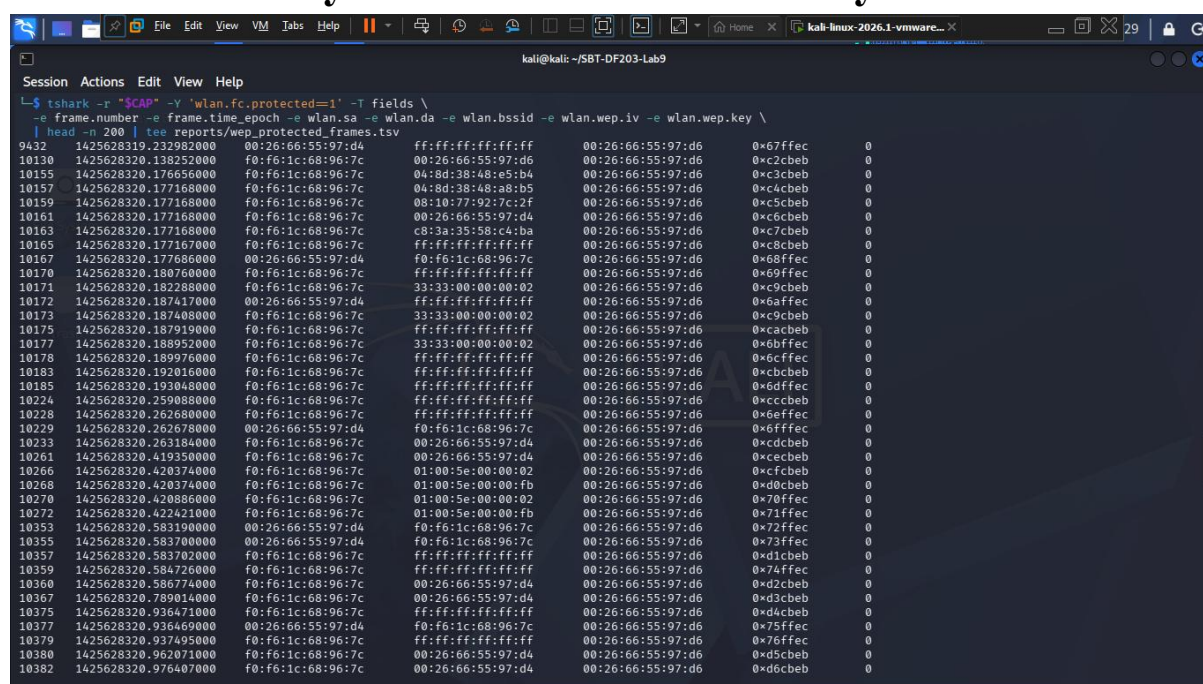
```
(kali@kali)~/SBT-DF203-Lab9
$ CAP=working/file_working
capinfos "$CAP" | tee reports/capinfos.txt
tshark -r "$CAP" -q -z io,phs | tee reports/protocol_hierarchy.txt
tshark -r "$CAP" -Y 'wlan' -T fields \
  -e frame.number -e frame.time -e wlan.fc.type -e wlan.fc.subtype -e wlan.sa -e wlan.da -e wlan.bssid \
  | head -n 100 | tee reports/wlan_frame_sample.tsv
File name: working/file_working
File type: Wireshark/tcpdump/... - pcap
File encapsulation: IEEE 802.11 Wireless LAN
File timestamp precision: microseconds (6)
Packet size limit: file hdr: 65535 bytes
Number of packets: 45 k
File size: 14 MB
Data size: 13 MB
Capture duration: 272.969795 seconds
Earliest packet time: 2015-03-06 08:51:54.364525
Latest packet time: 2015-03-06 08:56:27.334320
Data byte rate: 49 kbps
Data bit rate: 395 kbps
Average packet size: 299.13 bytes
Average packet rate: 165 packets/s
SHA256: c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d
SHA1: cb929fd6e407c13639e3168b0f35f921788478d3
Strict time order: False
Number of interfaces in file: 1
Interface #0 info:
  Encapsulation = IEEE 802.11 Wireless LAN (20 - ieee-802-11)
  Capture length = 65535
  Time precision = microseconds (6)
  Time ticks per second = 1000000
  Number of stat entries = 0
  Number of packets = 45169

Protocol Hierarchy Statistics
Filter:
frame frames:45169 bytes:13511274
wlan frames:45169 bytes:13511274
data frames:15713 bytes:13104128
```

Fig 9: Details of the Wireless Capture

Frame Category	Type Value	Purpose	Example Evidence
Management	0	Association, authentication, beacons	 Frame #54 (Beacon frame from 00:26:66:55:97:d6 to broadcast)
Control	1	Flow control and acknowledgements	 Frame #1, Block Ack (subtype 0x001b)
Data	2	Carries network payload	 Frame #9432 (Data frame from 00:26:66:55:97:d4 to broadcast)

Part B - Identify WEP Protection and Key Parameters



```
kali@kali: ~/SBT-DF203-Lab9
tshark -r "$CAP" -Y 'wlan.fc.protected==1' -T fields \
-e frame.number -e frame.time_epoch -e wlan.sa -e wlan.da -e wlan.bssid -e wlan.wep.iv -e wlan.wep.key \
| head -n 200 | tee reports/wep_protected_frames.tsv
9432 1425628319.23982000 00:26:66:55:97:d4 ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0x67ffec 0
10130 1425628320.138252000 f0:f6:1c:68:96:7c 00:26:66:55:97:d6 00:26:66:55:97:d6 0xc2cbeb 0
10155 1425628320.176656000 f0:f6:1c:68:96:7c 04:8d:38:48:e5:b4 00:26:66:55:97:d6 0xc3cbeb 0
10157 1425628320.177168000 f0:f6:1c:68:96:7c 04:8d:38:48:a8:b5 00:26:66:55:97:d6 0xc4cbeb 0
10159 1425628320.177168000 f0:f6:1c:68:96:7c 08:10:77:92:7c:2f 00:26:66:55:97:d6 0xc5cbeb 0
10161 1425628320.177168000 f0:f6:1c:68:96:7c 00:26:66:55:97:d4 00:26:66:55:97:d6 0xc6cbeb 0
10163 1425628320.177168000 f0:f6:1c:68:96:7c c8:3a:35:58:c4:ba 00:26:66:55:97:d6 0xc7cbeb 0
10165 1425628320.177167000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xc8cbeb 0
10167 1425628320.177686000 00:26:66:55:97:d4 f0:f6:1c:68:96:7c 00:26:66:55:97:d6 0xc8ffec 0
10170 1425628320.180760000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xc9ffec 0
10171 1425628320.182288000 f0:f6:1c:68:96:7c 33:33:00:00:00:02 00:26:66:55:97:d6 0xc9cbeb 0
10172 1425628320.187417000 00:26:66:55:97:d4 00:26:66:55:97:d6 00:26:66:55:97:d6 0xc6affec 0
10173 1425628320.187408000 f0:f6:1c:68:96:7c 33:33:00:00:00:02 00:26:66:55:97:d6 0xc9cbeb 0
10175 1425628320.187919000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xcacbeb 0
10177 1425628320.188952000 f0:f6:1c:68:96:7c 33:33:00:00:00:02 00:26:66:55:97:d6 0xc6ffec 0
10178 1425628320.189976000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xc6cfeb 0
10183 1425628320.192016000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xc6cbeb 0
10185 1425628320.193048000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xc6dfeb 0
10224 1425628320.259088000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xc6cbeb 0
10228 1425628320.262680000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xc6ffec 0
10229 1425628320.262678000 00:26:66:55:97:d4 00:26:66:55:97:d6 00:26:66:55:97:d6 0xc6ffec 0
10233 1425628320.263184000 f0:f6:1c:68:96:7c 00:26:66:55:97:d4 00:26:66:55:97:d6 0xc6cbeb 0
10261 1425628320.419350000 f0:f6:1c:68:96:7c 00:26:66:55:97:d4 00:26:66:55:97:d6 0xc6cbeb 0
10266 1425628320.420374000 f0:f6:1c:68:96:7c 01:00:5e:00:00:02 00:26:66:55:97:d6 0xc6cbeb 0
10268 1425628320.420374000 f0:f6:1c:68:96:7c 01:00:5e:00:00:02 00:26:66:55:97:d6 0xc6cbeb 0
10270 1425628320.420886000 f0:f6:1c:68:96:7c 01:00:5e:00:00:02 00:26:66:55:97:d6 0xc70ffec 0
10272 1425628320.422421000 f0:f6:1c:68:96:7c 01:00:5e:00:00:02 00:26:66:55:97:d6 0xc71ffec 0
10353 1425628320.583190000 00:26:66:55:97:d4 f0:f6:1c:68:96:7c 00:26:66:55:97:d6 0xc72ffec 0
10355 1425628320.583700000 00:26:66:55:97:d4 f0:f6:1c:68:96:7c 00:26:66:55:97:d6 0xc73ffec 0
10357 1425628320.583702000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xc71cbeb 0
10359 1425628320.584726000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xc74ffec 0
10360 1425628320.586774000 f0:f6:1c:68:96:7c 00:26:66:55:97:d4 00:26:66:55:97:d6 0xc72cbeb 0
10367 1425628320.789014000 f0:f6:1c:68:96:7c 00:26:66:55:97:d4 00:26:66:55:97:d6 0xc73cbeb 0
10375 1425628320.936471000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xc74cbeb 0
10377 1425628320.936469000 00:26:66:55:97:d4 f0:f6:1c:68:96:7c 00:26:66:55:97:d6 0xc75ffec 0
10379 1425628320.937495000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xc76ffec 0
10380 1425628320.962071000 f0:f6:1c:68:96:7c 00:26:66:55:97:d4 00:26:66:55:97:d6 0xc75cbeb 0
10382 1425628320.976407000 f0:f6:1c:68:96:7c 00:26:66:55:97:d4 00:26:66:55:97:d6 0xc76cbeb 0
```

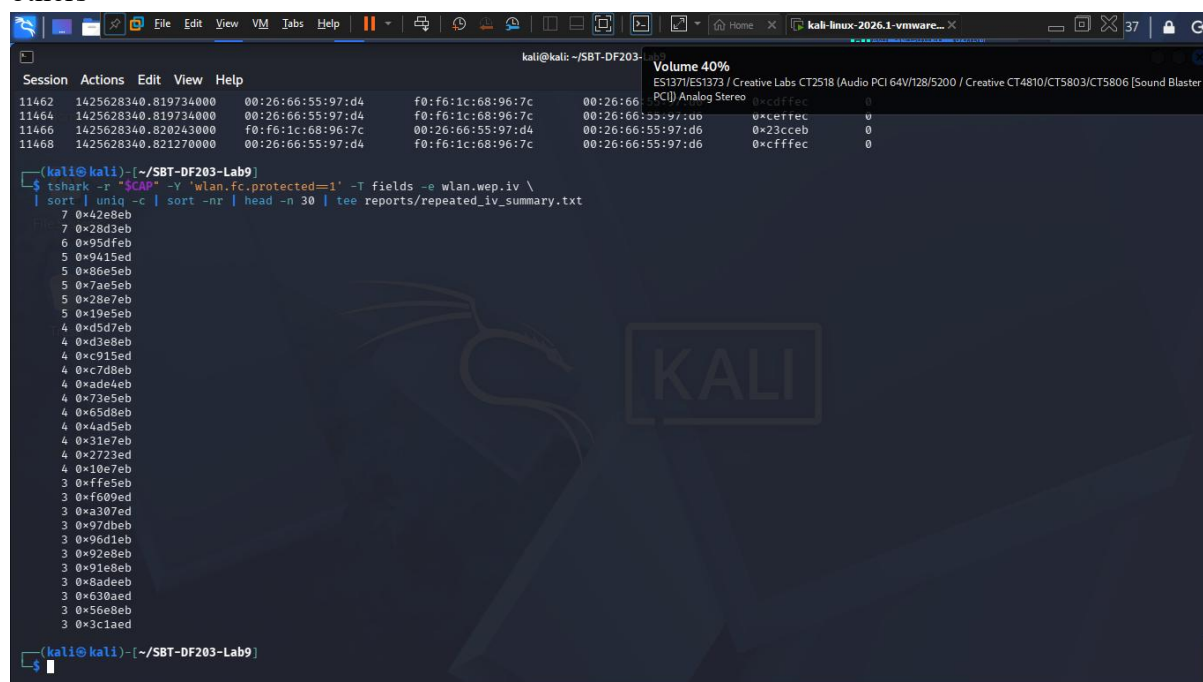
Fig 10: WEP Protection and Key Parameters

Here the Data Frames are confirmed, as the syntax captured 200 matching frames which was truncated by the flags all carrying non-empty WEP IV values and a wlan.wep.key index of 0.

The BSSID (access point): 00:26:66:55:97:d6

Common source MAC: f0:f6:1c:68:96:7c, a client device actively transmitting encrypted traffic

Sample IV values observed: 0x67ffec, 0xc2cbeb, 0xc3cbeb, 0xc4cbeb, 0xc5cbeb and many others



```
kali@kali: ~/SBT-DF203-Lab9
tshark -r "$CAP" -Y 'wlan.fc.protected==1' -T fields -e wlan.wep.iv \
| sort | uniq -c | sort -nr | head -n 30 | tee reports/repeated_iv_summary.txt
7 0x42e8eb
7 0x28d3eb
6 0x95dfef
5 0x9415ed
5 0x86e5eb
5 0x7ae5eb
5 0x28e7eb
5 0x19e5eb
4 0xd5d7eb
4 0xd3e8eb
4 0xc915ed
4 0xc7d8eb
4 0xade4eb
4 0x73e5eb
4 0x63d8eb
4 0x4ad5eb
4 0x31e7eb
4 0x2723ed
4 0x10e7eb
3 0xffe5eb
3 0xf609ed
3 0xa307ed
3 0x97d0eb
3 0x96d1eb
3 0x92e8eb
3 0x91e8eb
3 0x8adeeb
3 0x63daed
3 0x56e8eb
3 0x3c1aed
```

Fig 11: Fields and IV Numbers

```
(kali@kali)~[~/SBT-DF203-Lab9]
$ aircrack-ng "$CAP" | tee reports/aircrack_output.txt
Reading packets, please wait...
Opening working/file_working
Read 45169 packets.

# BSSID      ESSID      Encryption
1 00:26:66:55:97:D6  cgnetwork  WEP (15477 IVs)

Choosing first network as target.
Reading packets, please wait...
Opening working/file_working
Read 45169 packets.

1 potential targets
Attack will be restarted every 5000 captured ivs.

Aircrack-ng 1.7

[00:00:00] Tested 83 keys (got 15477 IVs)

KB  depth  byte(vote)
0  0/ 1  A4(22784) 62(20992) A8(19968) B6(19968) 42(19456) 6E(19456) 91(19200) 87(19200) 26(18944) 68(18944) 9E(18944) 0E(18688) 74(18688)
1  0/ 1  3D(23040) 51(20736) 07(20480) 62(19968) 78(19968) 1F(19712) B0(19712) 8D(19456) 80(19200) 85(19200) 9D(19200) 13(18944) EC(18944)
2  0/ 1  F6(23808) E4(20992) D0(20736) 68(20224) 95(19712) 38(19456) 0C(19200) 45(18944) 4F(18944) A8(18944) F7(18944) 60(18688) 63(18688)
3  1/ 10  F3(20480) C5(19968) D0(19968) 3E(19712) 43(19456) 52(19456) B2(19456) 09(19456) 20(19456) 8F(19200) A0(19200) B0(19200) 04(18944)
4  6/ 9  01(19712) 20(19456) 3E(19456) 52(19456) 5C(19456) 90(19456) F9(19456) 45(18944) 85(18944) 95(18944) D3(18944) 3A(18688) 40(18688)

KEY FOUND! [ A4:3D:F6:F3:74 ]
Decrypted correctly: 100%
```

Fig 12:

Selected network/BSSID: 00:26:66:55:97:D6 (ESSID: cgnetwork)

Number of IVs: 15,477

Key length: 64-bit WEP (40-bit secret key + 24-bit IV)

Recovered key: A4:3D:F6:F3:74

Decryption accuracy: 100%

This confirms that the recovered key is correct and matches the Initial training key provided on the slide exactly.

In summary, Aircrack-ng successfully recovered the 64-bit WEP key (A4:3D:F6:F3:74) for network "cgnetwork" (BSSID 00:26:66:55:97:D6) using 15,477 captured IVs, with 100% decryption accuracy, confirming both the tool's effectiveness and WEP's fundamental vulnerability to IV-based key recovery attacks.

Part D - Decrypt the Capture Offline

```
(kali㉿kali)-[~/SBT-DF203-Lab9]
$ cd working

(kali㉿kali)-[~/SBT-DF203-Lab9/working]
$ airdecap-ng -w A43DF6F374 file_working | tee ../reports/airdecap_output.txt
Total number of stations seen      10
Total number of packets read      45169
Total number of WEP data packets  15477
Total number of WPA data packets   0
Number of plaintext data packets   0
Number of decrypted WEP packets   15477
Number of corrupted WEP packets    0
Number of decrypted WPA packets    0
Number of bad TKIP (WPA) packets   0
Number of bad CCMP (WPA) packets   0

(kali㉿kali)-[~/SBT-DF203-Lab9/working]
$
```

Fig 13: Decrypt the capture

Up next, in the figure below we check the files we have in the working folder and then we check the hash values of each of them to see if they differ

```
(kali㉿kali)-[~/SBT-DF203-Lab9/working]
$ cd ..

(kali㉿kali)-[~/SBT-DF203-Lab9]
$ find working -maxdepth 1 -type f -ls | tee reports/decrypted_files_inventory.txt
 787337  13904 -rw-rw-r--  1 kali    kali    14234002 Sep 25 16:10 working/file_working
 787455  12488 -rw-rw-r--  1 kali    kali    12784501 Sep 29 01:13 working/file_working-dec
 787336  12540 -rw-rw-r--  1 kali    kali    12837660 Sep 28 14:12 working/file_working.xz

(kali㉿kali)-[~/SBT-DF203-Lab9]
$ sha256sum working/* | tee reports/all_working_file_hashes.txt
c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d  working/file_working
167c91994c269777f9048227deb89882caf3cf3c763977f2059604f9a6a40b04  working/file_working-dec
dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6  working/file_working.xz

(kali㉿kali)-[~/SBT-DF203-Lab9]
$
```

Fig 14: Checking the Hash Value of all working files

And we see that they all have different hash values which mean they have different bit number and differ in some ways.

I then opened the decrypted file in wireshark and confirmed that both the IP/TCP/HTTP Protocols are found there

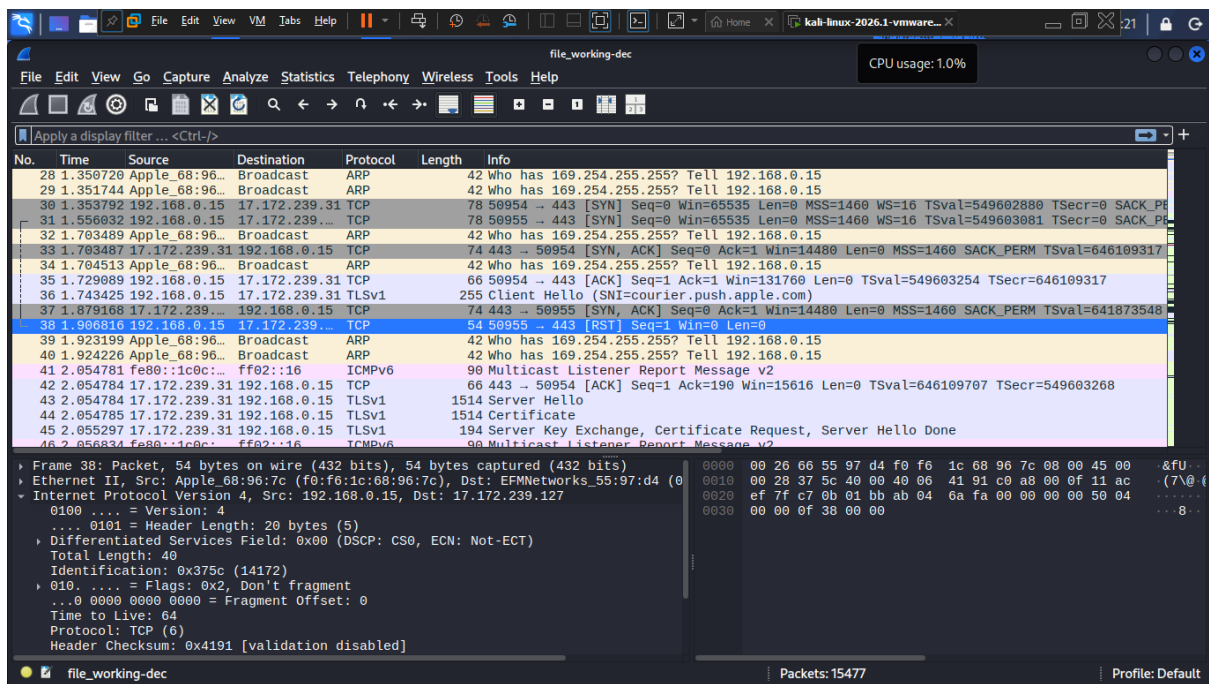


Fig 15: Wireshark Analysis of working/file_working-dec

Part E - Extract Endpoints and Conversations

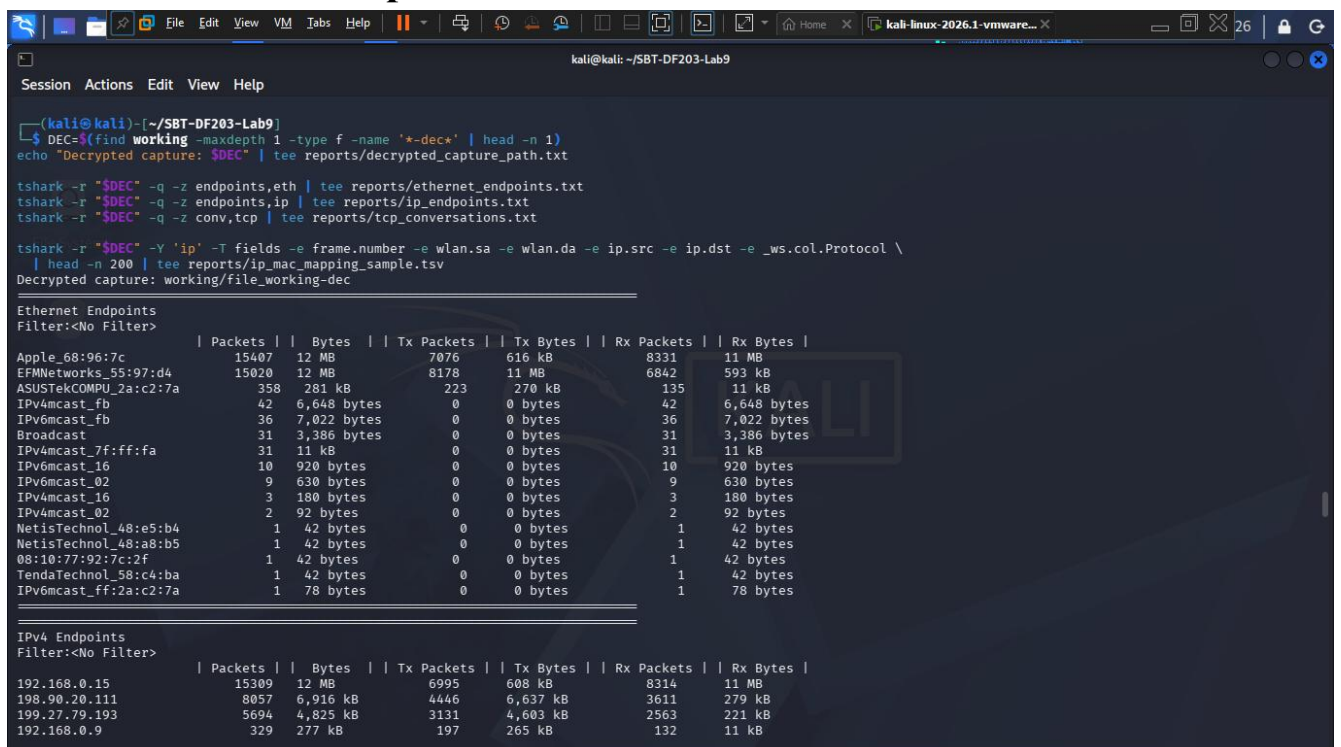
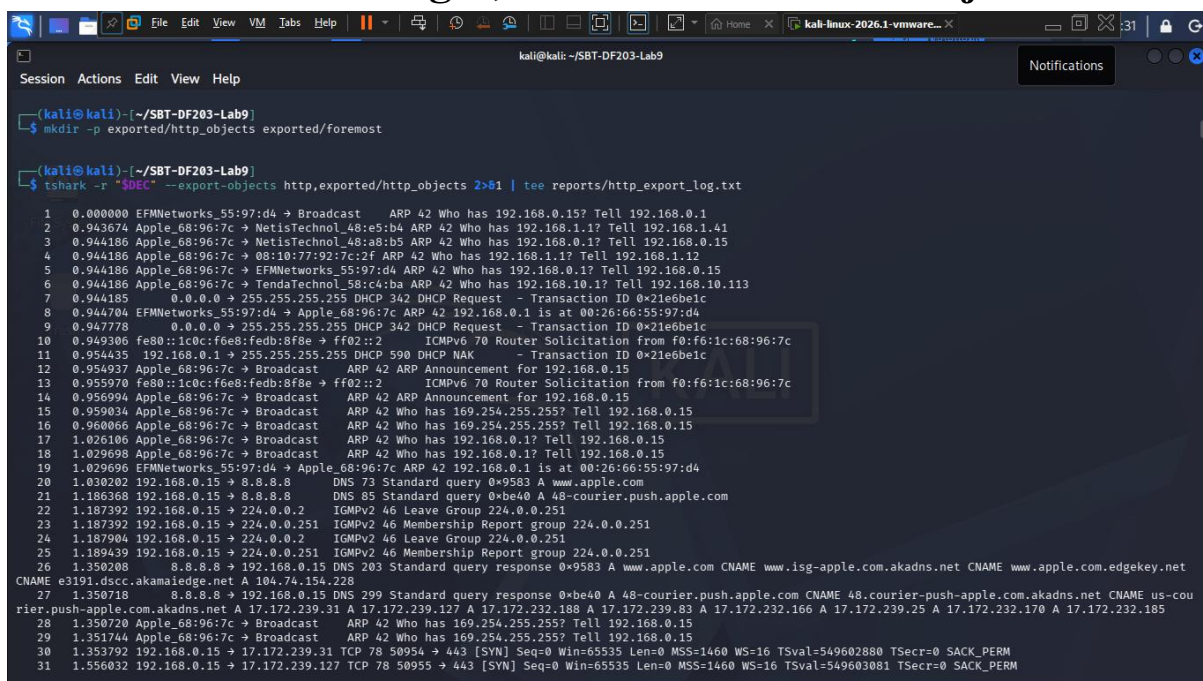


Fig 16: Extracting Endpoints and Conversations from each Protocol

Part F - Extract Images, HTML and Other Objects



```
kali@kali: ~/SBT-DF203-Lab9
Session Actions Edit View Help

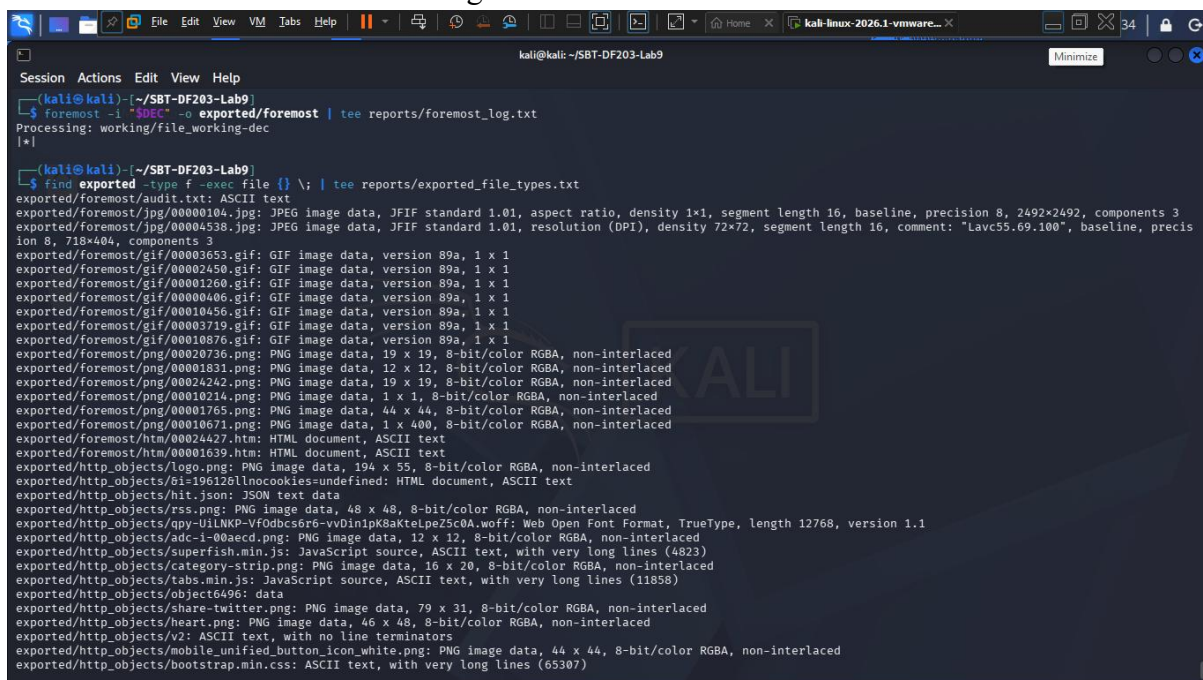
(kali@kali)~[~/SBT-DF203-Lab9]
$ mkdir -p exported/http_objects exported/foremost

(kali@kali)~[~/SBT-DF203-Lab9]
$ tshark -r "$DEC" --export-objects http,exported/http_objects 2>&1 | tee reports/http_export_log.txt

1 0.000000 EFMNetworks_55:97:d4 → Broadcast ARP 42 Who has 192.168.0.15? Tell 192.168.0.1
2 0.943674 Apple_68:96:7c → NetisTechnol_48:e5:b4 ARP 42 Who has 192.168.1.1? Tell 192.168.1.41
3 0.944186 Apple_68:96:7c → NetisTechnol_48:a8:b5 ARP 42 Who has 192.168.0.1? Tell 192.168.0.15
4 0.944186 Apple_68:96:7c → 08:10:77:92:7c:2f ARP 42 Who has 192.168.1.1? Tell 192.168.1.12
5 0.944186 Apple_68:96:7c → EFMNetworks_55:97:d4 ARP 42 Who has 192.168.0.1? Tell 192.168.0.15
6 0.944186 Apple_68:96:7c → TendaTechnol_58:c4:ba ARP 42 Who has 192.168.10.1? Tell 192.168.10.113
7 0.944185 0.0.0.0 → 255.255.255.255 DHCP 342 DHCP Request - Transaction ID 0x21e6be1c
8 0.944784 EFMNetworks_55:97:d4 → Apple_68:96:7c ARP 42 192.168.0.1 is at 00:26:66:55:97:d4
9 0.947778 0.0.0.0 → 255.255.255.255 DHCP 342 DHCP Request - Transaction ID 0x21e6be1c
10 0.949306 fe80::1c0c:f6e8:fedb:8f8e → ff02::2 ICMPv6 70 Router Solicitation from f0:f6:1c:68:96:7c
11 0.954435 192.168.0.1 → 255.255.255.255 DHCP 590 DHCP NAK - Transaction ID 0x21e6be1c
12 0.954937 Apple_68:96:7c → Broadcast ARP 42 ARP Announcement for 192.168.0.15
13 0.955970 fe80::1c0c:f6e8:fedb:8f8e → ff02::2 ICMPv6 70 Router Solicitation from f0:f6:1c:68:96:7c
14 0.956994 Apple_68:96:7c → Broadcast ARP 42 ARP Announcement for 192.168.0.15
15 0.959034 Apple_68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
16 0.960066 Apple_68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
17 1.026106 Apple_68:96:7c → Broadcast ARP 42 Who has 192.168.0.1? Tell 192.168.0.15
18 1.026998 Apple_68:96:7c → Broadcast ARP 42 Who has 192.168.0.1? Tell 192.168.0.15
19 1.029696 EFMNetworks_55:97:d4 → Apple_68:96:7c ARP 42 192.168.0.1 is at 00:26:66:55:97:d4
20 1.030202 192.168.0.15 → 8.8.8.8 DNS 73 Standard query 0x9583 A www.apple.com
21 1.186368 192.168.0.15 → 8.8.8.8 DNS 85 Standard query 0xb40 A 48-courier.push.apple.com
22 1.187202 192.168.0.15 → 224.0.0.2 IGMPv2 46 Leave Group 224.0.0.251
23 1.187392 192.168.0.15 → 224.0.0.251 IGMPv2 46 Membership Report group 224.0.0.251
24 1.187904 192.168.0.15 → 224.0.0.251 IGMPv2 46 Leave Group 224.0.0.251
25 1.189439 192.168.0.15 → 224.0.0.251 IGMPv2 46 Membership Report group 224.0.0.251
26 1.350208 8.8.8.8 → 192.168.0.15 DNS 203 Standard query response 0x9583 A www.apple.com CNAME www.isg-apple.com.akadns.net CNAME www.apple.com.edgekey.net
CNAME e3191.dccc.akamaiedge.net A 104.74.154.228
27 1.350718 8.8.8.8 → 192.168.0.15 DNS 299 Standard query response 0xb40 A 48-courier.push.apple.com CNAME 48.courier-push.apple.com.akadns.net CNAME us-cou
rier.push-apple.com.akadns.net A 17.172.239.31 A 17.172.239.127 A 17.172.232.188 A 17.172.239.83 A 17.172.232.166 A 17.172.239.25 A 17.172.232.170 A 17.172.232.185
28 1.350720 Apple_68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
29 1.351744 Apple_68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
30 1.353792 192.168.0.15 → 17.172.239.31 TCP 78 50954 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=16 TSval=549602880 TSecr=0 SACK_PERM
31 1.556032 192.168.0.15 → 17.172.239.127 TCP 78 50955 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=16 TSval=549603081 TSecr=0 SACK_PERM
```

Fig 17: Extracting Images and Other things using tshark

Now we want to carve the files using **foremost** command and then we discover them



```
kali@kali: ~/SBT-DF203-Lab9
Session Actions Edit View Help

(kali@kali)~[~/SBT-DF203-Lab9]
$ foremost -i "$DEC" -o exported/foremost | tee reports/foremost_log.txt
Processing: working/file_working-dec
[*]

(kali@kali)~[~/SBT-DF203-Lab9]
$ find exported -type f -exec file {} \; | tee reports/exported_file_types.txt
exported/foremost/audit.txt: ASCII text
exported/foremost/jpg/0000010a.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, baseline, precision 8, 2492x2492, components 3
exported/foremost/jpg/00004538.jpg: JPEG image data, JFIF standard 1.01, resolution (DPI), density 72x72, segment length 16, comment: "Lavc55.69.100", baseline, precis
ion 8, 718x404, components 3
exported/foremost/gif/00003653.gif: GIF image data, version 89a, 1 x 1
exported/foremost/gif/00002450.gif: GIF image data, version 89a, 1 x 1
exported/foremost/gif/00001260.gif: GIF image data, version 89a, 1 x 1
exported/foremost/gif/00000406.gif: GIF image data, version 89a, 1 x 1
exported/foremost/gif/00001045.gif: GIF image data, version 89a, 1 x 1
exported/foremost/gif/00003719.gif: GIF image data, version 89a, 1 x 1
exported/foremost/gif/00010876.gif: GIF image data, version 89a, 1 x 1
exported/foremost/png/00020736.png: PNG image data, 19 x 19, 8-bit/color RGBA, non-interlaced
exported/foremost/png/00001831.png: PNG image data, 12 x 12, 8-bit/color RGBA, non-interlaced
exported/foremost/png/00024242.png: PNG image data, 19 x 19, 8-bit/color RGBA, non-interlaced
exported/foremost/png/00010214.png: PNG image data, 1 x 1, 8-bit/color RGBA, non-interlaced
exported/foremost/png/00001765.png: PNG image data, 44 x 44, 8-bit/color RGBA, non-interlaced
exported/foremost/png/00010671.png: PNG image data, 1 x 400, 8-bit/color RGBA, non-interlaced
exported/foremost/htm/00024227.htm: HTML document, ASCII text
exported/foremost/htm/00001639.htm: HTML document, ASCII text
exported/http_objects/logo.png: PNG image data, 194 x 55, 8-bit/color RGBA, non-interlaced
exported/http_objects/gi-1961261lnocookies=undefined: HTML document, ASCII text
exported/http_objects/hit.json: JSON text data
exported/http_objects/rss.png: PNG image data, 48 x 48, 8-bit/color RGBA, non-interlaced
exported/http_objects/qpy-ViLNKP-VfOdbcs6r6-vDinIpK8aktelpeZ5c0A.woff: Web Open Font Format, TrueType, length 12768, version 1.1
exported/http_objects/adc-1-00aecd.png: PNG image data, 12 x 12, 8-bit/color RGBA, non-interlaced
exported/http_objects/superfish.min.js: JavaScript source, ASCII text, with very long lines (4823)
exported/http_objects/category-strip.png: PNG image data, 16 x 20, 8-bit/color RGBA, non-interlaced
exported/http_objects/tabs.min.js: JavaScript source, ASCII text, with very long lines (11858)
exported/http_objects/object6496: data
exported/http_objects/share-twitter.png: PNG image data, 79 x 31, 8-bit/color RGBA, non-interlaced
exported/http_objects/heart.png: PNG image data, 46 x 48, 8-bit/color RGBA, non-interlaced
exported/http_objects/v2: ASCII text, with no line terminators
exported/http_objects/mobile_unified_button_icon.white.png: PNG image data, 44 x 44, 8-bit/color RGBA, non-interlaced
exported/http_objects/bootstrap.min.css: ASCII text, with very long lines (65307)
```

Fig 18: Carving Files using Foremost

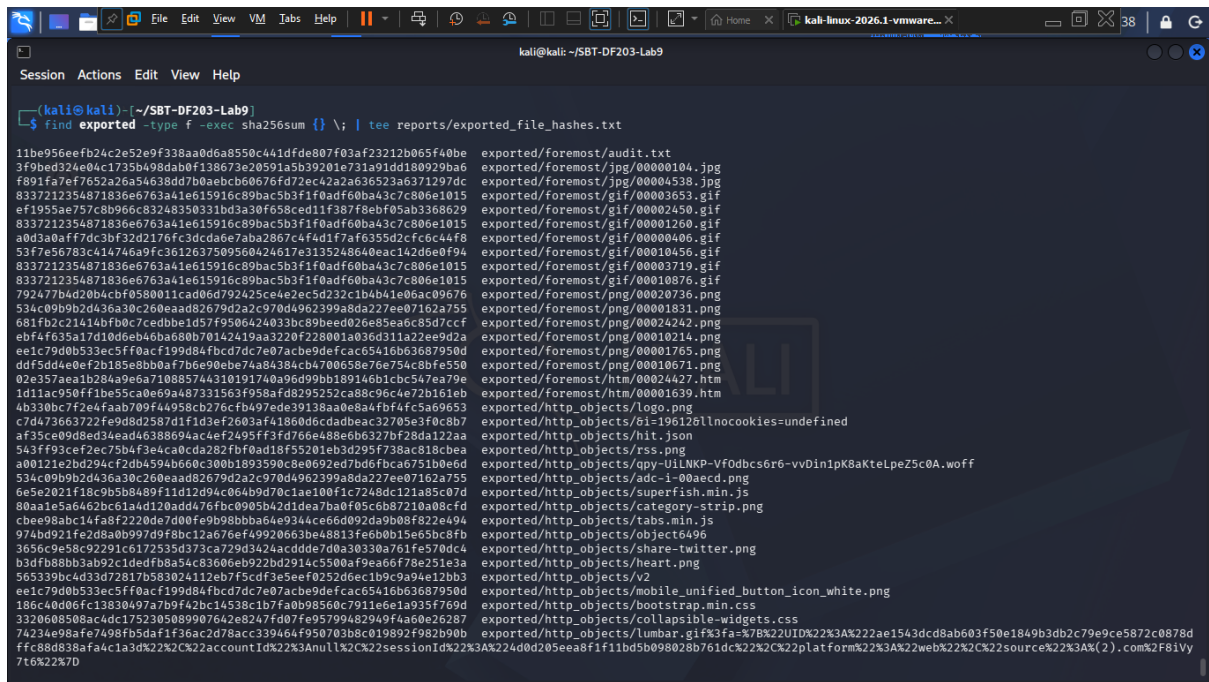


Fig 19: The Hash values for all the Exported files

Required Findings Worksheet

Question	Finding
Capture format and duration	Capture Format: IEEE 802.11 (Wireless LAN, pcap format) Duration: 272.969795 seconds (~4.5 minutes), Total Packets captured: 45,169 total packets
BSSID	00:26:66:55:97:D6 (ESSID: cgnetwork)
Primary station MAC addresses	f0:f6:1c:68:96:7c (primary client), 00:26:66:55:97:d4 and 00:26:66:55:97:d6 (access point-related addresses)
Protected frame count	200+ confirmed in sample and it was capped by the flag head -n 200
Repeated IV evidence	Multiple IVs reused within the capture, most notably 0x42e8eb and 0x28d3eb with them appearing over 5 times.
Recovered/validated WEP40 key	A4:3D:F6:F3:74 Recovered using Aircrack-ng with a 100% decryption accuracy.
Decrypted capture filename/hash	Filename: working/file_working-dec SHA-256: 167c91994c269777f9048227deb89882caf3cf3c763977f2059604f9a6a40b04
Host IP and MAC	Host Ip: 192.168.15 Host MAC: f0:f6:1c:68:96:7c

Question	Finding
Server IP and MAC	Server MAC: 00:26:66:55:97:d6 Server IP: 198.90.20.111
Key protocols observed	HTTP, TCP,DNS, ARP, IP
Recovered images/HTML	
Optional router passphrase	Protected appendix only
Security conclusion	In under 5 minutes of captured traffic, I was able to fully recover the WEP key using aircrack-ng with 100% decryption accuracy. This showed me firsthand how weak WEP really is, the repeated IVs I found earlier made the key recovery possible almost effortlessly. WEP clearly isn't safe for protecting any real wireless network today.